

[P100] Install

- Description
- P100 Configuration
 - SystemrescueCD setup
 - Save server image
- Server restore
 - Configurations
 - Teensy configuration (screen in the server) → Playbook Install firmware scripts
 - SIM configuration
 - Install grafana sensors (if required):
 - Check Crew App (ONLY for CAL)
- Others features

Task owner	Ops
Time duration	2h
Escalation	

Description

Steps to setup a new P100 server.

P100 Configuration

SystemrescueCD setup

1. https://git.immfly.com/immfly/restore_server/blob/master/docs/BuildSystemp100_OperP100_Operations_ManualRescueCD.md

Remove the front panel and back panel (with screwdriver). Plug the **usbrescuecd** to the front usb port and the serial converter to the back serial port.

Save server image

Before the save you have to perform the following steps:

i The access point is, by default, accessible through the IP address **192.168.1.1**

Run bash script at `/root/p100_tools/wifi_router/restore_bkp.sh` that restores a backup copy to the access point. Then, the AP is accessible through the IP address 192.168.150.203.

```
1 | /root/p100_tools/wifi_router/restore_bkp.sh
```

i Then, the AP is accessible through the IP address **192.168.150.203**.

Go to 192.168.150.203 to change the AP SSID (`/etc/config/wireless`) and save a config backup by running `/root/p100_tools/wifi_router/generate_bkp.sh`.

```
1 | /root/p100_tools/wifi_router/generate_bkp.sh
```

It generates a backup file under `/root/p100_tools/wifi_router/`

Modify the startup file `/etc/default/routerconfig` to point to the newly created backup
(line 3)

```
1 [VIV:XA-007] root@p100-ify:~# cat /etc/default/routerconfig
2 ROUTERIP="192.168.1.1"
3 DAEMON_ARGS="/root/p100_tools/wifi_router/backup_config-OpenWrt-2022-03-08_2003_47.tar.gz"
4 [VIV:XA-007] root@p100-ify:~#
```

And apply the changes

```
1 | /etc/init.d/routerconfig install
```

Be sure that file “**havetorestore**” is present in the following directory:

```
1 | touch /root/p100_tools/teensy_kits/havetorestore
```

Do a **Filesync clean** before start SAVE process

⚠ Add link to filesync

To manually execute syncs (first stop flight-controller, autopilot off and vm_restart comment on crontab) \$ autopilot off \$ autopilot ssh \$ cd /immfly/srv \$ docker-compose stop flight-controller

Dump/Air Sync: \$ docker-compose exec aircraft ./manage.py airdsync sync --force-reset

Content: \$ docker-compose exec aircraft ./manage.py filesync run -ct 16 -st 16 -cv -sv

Clean: \$ docker-compose exec aircraft ./manage.py filesync clean Logs: docker-compose logs -f --tail=1000 flight-controller

Finally:

- 1 | \$ docker-compose up -d flight-controller

Server restore

Open the back panel and connect the **serial cable/usb converter** to the serial port and usb port to your laptop.

Open the front panel and connect the **systemrescuecd** to the usb port

Turn on the P100 server and press **SUPR** key to enter bios configuration. Choose the USB boot option and use the bootable option with TTY serial display.

When ready, remove the systemrescuecd and plug the hard drive with the p100 image and do the following commands.

Check the disk you want to mount

```
1 NAME          MAJ:MIN RM   SIZE RO TYPE MOUNTPOINT
2 sda             8:0    1  28.9G  0 disk
3 └─sda1          8:1    1   2.8G  0 part
4 └─nvme0n1       259:0    0 953.9G  0 disk
```

In this case is sda1

```
1 $ mkdir /mnt/usb
2 $ mkdir /mnt/custom
3 $ mount /dev/sda1 /mnt/usb
4 $ cd /mnt/usb/p100_restore_EFI
5 $ python3 save_server.py save nvme0n1
```

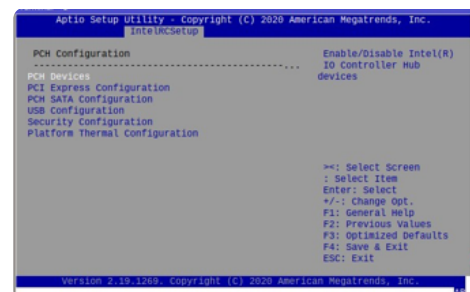
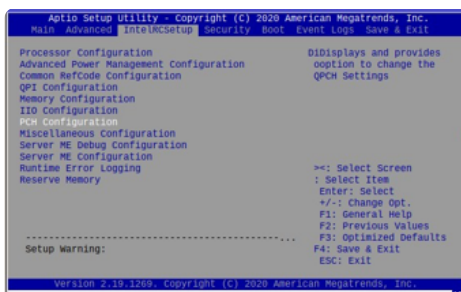
Check the partitions created by the script.

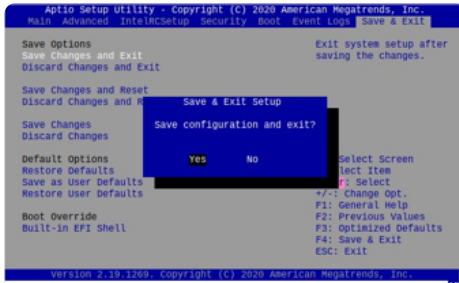
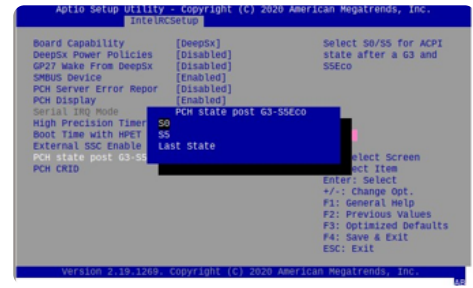
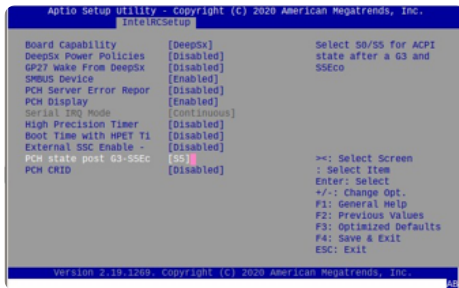
```
1 [VIV:XA-007] root@p100-ify:~/p100_tools/wifi_router# lsblk
2 NAME          MAJ:MIN RM   SIZE RO TYPE MOUNTPOINT
3 loop0           7:0     0    19G  0 loop
4 nvme0n1        259:0    0 953.9G  0 disk
5 └─nvme0n1p1    259:1     0   190M  0 part /boot/efi
6 └─nvme0n1p2    259:2     0   25.1G  0 part /
7 └─nvme0n1p3    259:3     0   29.8G  0 part
8 └─nvme0n1p4    259:4     0   41.9G  0 part /kvmroot
9 └─nvme0n1p5    259:5     0  368.7G  0 part /mnt/wwwdata
```

Reboot Server and press **SUPR to enter to bios** and configure the following parameter:

For P100 HW 1.0 / 1.1

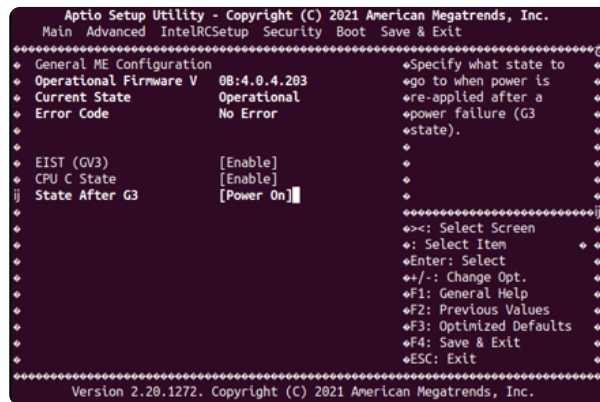
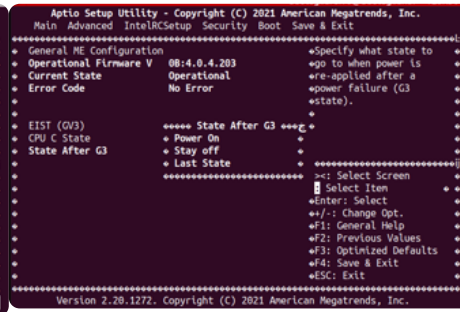
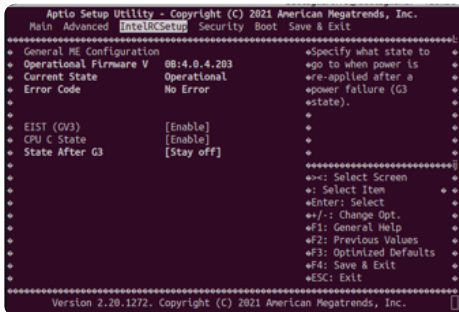
1. IntelRCSetup → PCH Configuration → PCH Devices → PCH state post G3-S5Ec ;
Set this from “S5” to “**SO**”
2. Save and Exit
3. Reboot and check our system is booting





For P100 HW 3.0

1. IntelRCSetup → State After G3 → Set this to “Power On”
2. Save and Exit
3. Reboot and check our system is booting





Configurations

Connect via ethernet wire to the server

```
1 | ssh root@192.168.150.202 -p 32765
```

If after the server restore there is no ethernet connection, contact infra.

ⓘ If after restore there is no wifi listed for the p100:

Go to **/root/p100_tools/wifi_router** and check that the last configuration backup is listed. If not, place the corresponding backup from a repository (select the one for the airline) and then run:

```
# ./restore_bkp.sh backup...
```

If no clock

```
1 | ntpdate -stime.nist.gov
```

Add the aircraft to fleet api (+ ansible inventory), hangar node and hangar airplanes and create a VPN host in both servers (torredecontrol and tdc-temp) following the document below, point **06**

List of Platforms that New AC's should be inserted

<https://docs.google.com/document/d/1JET59odPVrsFdYY5NW2juDAA71VmWRPYWcrJ5kekK18/edit?pli=1#heading=h.ouosf7rle82> **Connect your Google Drive account**

ⓘ To connect the server to internet from the office

```
route add default gw 192.168.150.1
```

Now the basic configuration can be applied:

```
1 | VPNUSER=
2 | PLACEICA0=
3 | AIRLINE=
```

VPNUSER: server hostname (same as vpn username)

planeicao: hex code from flightradar TODO: link document

airline: 3 digit code for airline

```
1 autopilot off
2 echo "$VPNUSER" > /etc/planename
3 echo "$PLANEICA0" > /etc/planeicao
4 echo "$AIRLINE" > /etc/airline
```

Check if the values are right in the following file (lines 6, 13, 14, 15)

```
1 cat /var/lib/lxc/autopilot/rootfs/etc/environment
2 PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games"
3 AUTOPILOT_SETTINGS=settings.pro
4 VM_HYPERVISOR=qemu:///system
5 AUTOPILOT_VM_CPU=3
6 AIRLINE=VIV
7 AIRSYNC_ENABLED=True
8 APP_BASIC_AUTH_PASSWORD=jABKMndXqX9L
9 APP_BASIC_AUTH_USERNAME=immfly
10 CHECK_TIMEOUT_PORT=450
11 COMPOSE_PROJECT_NAME=ify
12 DJANGO_SETTINGS_MODULE=settings.air.pro.fleet
13 HOST_DOMAIN=immfly.com
14 HOST_SUBDOMAIN=air
15 PLANENAME=XA-007
16 SOURCE_AUTH_PASSWORD=XXXXXXXXXX
17 SOURCE_AUTH_USERNAME=autopilot
```

 Update the variables value accordingly to the airline info.

Apply the same changes inside the VM

```
1 autopilot ssh
2 VPNUSER=
3 AIRLINE=
4 sudo echo "$VPNUSER" > /etc/planename
5 sudo echo "$AIRLINE" > /etc/airline
```

Check if the values are right in the following file (lines 6, 13, 14, 15)

```
1 cat /etc/environment
2 PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games"
3 AUTOPILOT_SETTINGS=settings.pro
4 VM_HYPERVISOR=qemu:///system
5 AUTOPILOT_VM_CPU=3
6 AIRLINE=VIV
7 AIRSYNC_ENABLED=True
8 APP_BASIC_AUTH_PASSWORD=jABKMndXqX9L
9 APP_BASIC_AUTH_USERNAME=immfly
10 CHECK_TIMEOUT_PORT=450
11 COMPOSE_PROJECT_NAME=ify
12 DJANGO_SETTINGS_MODULE=settings.air.pro.fleet
13 HOST_DOMAIN=immfly.com
14 HOST_SUBDOMAIN=air
15 PLANENAME=XA-007
16 SOURCE_AUTH_PASSWORD=XXXXXXXXXX
17 SOURCE_AUTH_USERNAME=autopilot
```

Setup VPN credentials provided by OPS (auth.cfg)

```
1 vim /etc/openvpn/auth.cfg
2 service openvpn restart
```

Reboot, connect again to the internet with the gateway and deploy the release in apapi if required.

Add the aircraft host to Darom AWX

Teensy configuration (screen in the server) → Playbook *Install firmware scripts*

⚠ We should have previously configured the planename and airline parameters.

We need to setup the following envvars:

```
1 TEENSY_VERSION=XXXX
2 GITLAB_TOKEN=XXXX
```

Then the firmware associated with the hardware revision is launched by the following playbook command:

```
1 ansible-playbook ./playbooks/fleets/install_scripts_p100.yml -u root -i
  192.168.150.202:32765, -e "gitlab_token=$GITLAB_TOKEN firmware_revision=$TEENSY_VERSION" -vv
```

TODO: review the command above

The token changes during time → check Infra.

⚠ Check that the functionalities in the screen are correct as requirements.

From OPS the way to configure the teensy is via DAROM with following variables

```
1 gitlab_token: gitlab token to download projects and artifacts (Default).
2 firmware_revision: $TEENSY_VERSION
3 planeicao: $PLANEICAO
4 disabled_checktemp: 'no'
5 # 'yes' to deactivate the automatic shutdown when the maximum temperature is reached.
6 cpu_maxtemp: 80000
7 teensy_maxtemp: 60000
8 nvme0_maxtemp: 70000
9 bno055_maxtemp: 60000
10 altitude: ADSB altitude for wifi automatic on (only China)
11 no_log: false
12 # true if we need to check for errors
```

SIM configuration

In order to check the connectivity with modem and SIM card, just do the following modification in line 33 on this file depending on the airline.

```
1 cat /etc/ppp/peers/gsm_chat-mc8795v
2
```

```

3 #####
4 Connection script for Sierra Wireless GSM/UMTS modems
5 Note: This demo script is setup to work on the Cingular EDGE network
6 #
7 TIMEOUT 10
8 SAY '\nCHAT: Starting Sierra Wireless GPRS/UMTS connect script...\n'
9
10 #####
11 SAY 'CHAT: Setting the abort string\n'
12 Abort String -----
13 ABORT 'NO DIAL TONE' ABORT 'NO ANSWER' ABORT 'NO CARRIER' ABORT DELAYED
14 #####
15
16 #####
17 SAY 'CHAT: Initializing modem\n'
18 Modem Init
19 '' AT
20 OK ATZ
21 #####
22
23 #####
24 SAY 'CHAT: Setting Access Point Name (APN)\n'
25
26 Access Point Name (APN)
27 Incorrect APN or CGDCONT can often cause errors in connection.
28 Below are the possible AT&T APNs we know about.
29 Test APN
30 #OK 'AT+CGDCONT=1,"IP","m2m.orange.es"'
31
32 Production APN
33 OK 'AT+CGDCONT=1,"IP","wwcorps.itelcel.com"'
34 #####
35
36 #####
37 SAY 'CHAT: Dialing ISP...\n\n'
38 Dial the ISP, this is the common Cingular dial string
39 OK ATD*99#
40 CONNECT ''
41 #####

```

Airline	APN
CAL	OK 'AT+CGDCONT=1,"IP","internet"'
VIVA	OK 'AT+CGDCONT=1,"IP","fast.t-mobile.com"'
ETH	OK 'AT+CGDCONT=1,"IP","etc.com"'
VOE	OK 'AT+CGDCONT=1,"IP","m2m.orange.es"'
	or
	OK 'AT+CGDCONT=1,"IP","m2m.nat.es"'
AVA	OK 'AT+CGDCONT=1,"IP","internet.movistar.com.co" '

APN	OK 'AT+CGDCONT=1,"IP","ac.vodafone.es"'
Vodafone	

Turn off the server and place the SIM card carefully. Turn on the server, and check that there is connection with:

```

1 tail -f /var/log/internet_ppp.log
2
3 2023-01-11 10:57:01,093 INFO: ADS-B is not connected and plane is pretty quiet
4 2023-01-11 10:57:01,098 INFO: PPP status: 0
5 2023-01-11 10:57:04,134 INFO: PPP status: 0

```

✖ If INFO: PPP status: 1 review the modem status.

WAP & SIM Config

 P100_config.pdf  upgrade_wap.sh

Check the crontab and check **internet_p100.py** is uncommented (line 6 in the case below)

```

1 cat /etc/crontab
2
3 /etc/crontab: system-wide crontab
4 ...
5 #Ansible: internet
6 root /usr/bin/python /root/p100_tools/internet/internet-p100.py
7
8 #Ansible: checktemp
9 ## * * * * root /root/v-scripts/temperature/check_temp.sh -z thermal_zone0:75000 -z
10 teensy:65000 -z nvme0:70000 -z bno055:90000
11
12 #Ansible: movement
13 ## * * * * root /usr/bin/python /root/p100_tools/movement/check_movement.py --max-accel-
14 value 1.0
15
16 #Ansible: states_actions
17 @reboot root /root/v-scripts/states_actions.sh
18
19 #Ansible: execute sensors
20 root /usr/bin/python3 /scripts/collector.py
21
22 #Ansible: battery-p100
23 ## * * * * root /usr/bin/python /root/p100_tools/battery/battery_p100.py --min 500
24 ## * * * * root /root/reboot_test.sh

```

Install grafana sensors (if required):

- Install the sensors using playbook template with variables for p100
 - Playbook: Deploy sensors POC
- Install the temperature playbook:
 - Playbook: Set P100 temperature

- Check Teensy temperature is collecting and delivering data
- Add the aircraft in Grafana and update Influx is needed (TODO)

Check Crew App (ONLY for CAL)

Check that these services are UP

```
ify_crewapi_1      /bin/sh      Up    8000/tcp
```

```
                /exec/bootstrap.sh
```

```
ify_crewapp_1      nginx -g daemon off;  Up    80/tcp
```

In **/etc/crontab**

#Ansible: states_actions

@reboot root /root/v-scripts/states_actions.sh

✓ TO REVIEW

Others features

- Update *internet-p100.py* script in */root/p100_tools/internet/internet-p100.py*
- ADS-B
 - Remove screen from */etc/init.d/rc.local*
 - Install Python packages
- Cannot power off the server due to high temperature through a BIOS option
- The server will power off if the CPU or screen temperature reaches 80 degrees
 - Can be configured in */etc/crontab*
- The server cannot be powered off by pressing an external button
- Not used buttons display INOP in the screen
- Airline name on the screen is now not hardcoded. Can be changed in ***/etc/teensytitle***
- Wifi action definition ***/etc/teensyconfig*** (JSON dictionary)

```
{
  "buttons": {
    "1": [
```

```
    "inop"
  ],
  "2": [
    "wifi_on",
    "wifi_off",
    "wifi_auto"
  ],
  "3": [
    "internet"
  ],
  "4": [
    "inop"
  ],
  "5": [
    "inop"
  ]
},
"callbacks": {
  "wifi_off": "led2off",
  "wifi_on": "led2on"
},
"wap": "auto",
"title": "China Airlines"
}
```

Check firmware revision:

```
$ python3 -c 'from teensycmds.teensycmds import TeensyBoard;
t=TeensyBoard();print(t.cmd("ping"))'
```

Check SN:

```
$ sudo dmidecode -t system | grep Serial
```

OR

```
$ sudo dmidecode -s 'chassis-serial-number'
```

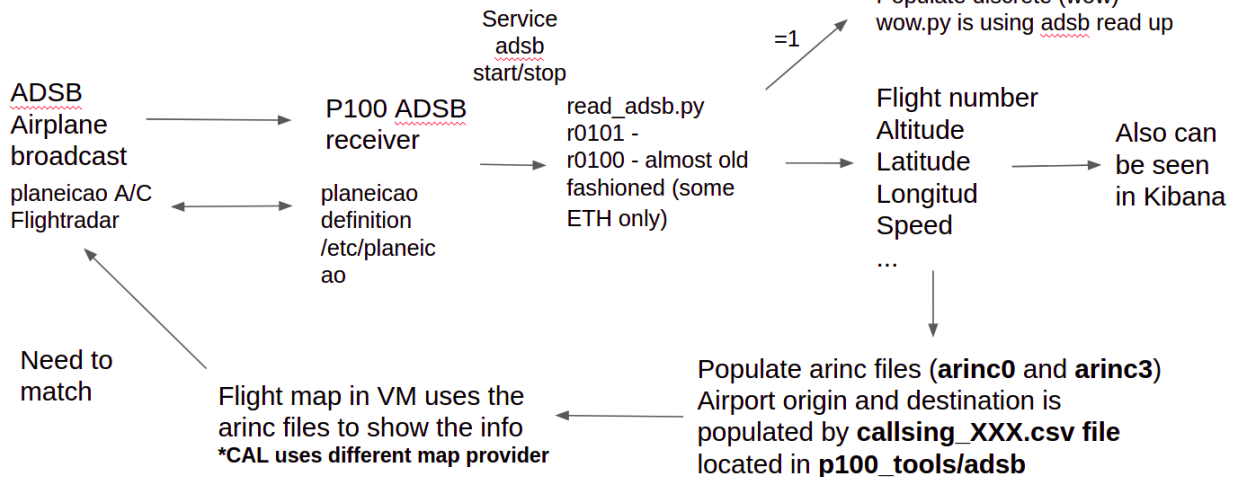
Read ADSB service:

```
$ ps aux | grep adsb
```

```
$ service adsb status
```

Dataflow

*arinc1 and arinc2 must be empty for
p100 (only works for embedded)



In order to modem start connection it checks a file in /tmp/, called adsb-connected. If the file exists, the modem connection is over. If not, modem starts connections.

Internet script checks the file and the adbs script creates and removes the file.

IMPORTANT: in ETH we don't care about ADSB.

The python script is run here:

start on startup

script

cd /root/p100_tools/adsb

while ;; do

sleep 10s

/usr/bin/python ./read_adsb.py

done

end script

Python3 and Python compatibility

Some of the data taken from Sensors should connect to the teensy via serial. Be aware that the **pyserial** version corresponds to the working one.

If some data from sensors-teensy is no visible run the following commands:

\$ pip list | grep seri

\$pip3 list | grep seri

If needed run:

pip3 install pyserial==3.4

Check Temperature thresholds:

grep check_temp /etc/crontab

Debug mode in ADSB

Create a file in /etc/default/ called "adsb"

Add this line to the file and save

LOGLEVEL=DEBUG

Finally run this command

Service adsb restart

Removing arinc1 and arinc2 content

We need to remove the content for arinc2 and arinc 1 in the P100 boxes

for ch in 1 2; do > /states/arinc\${ch}; done

Check Battery status (last firmware update 9/aug)

```
python -c 'from teensycmds.teensycmds import  
TeensyBoard;t=TeensyBoard();print(t.cmd("battery"))'
```

- Auto switch on when power signal is ON
- Auto switch off when battery reach 10%
- Battery loading manager
- Sensors module in sensors project to check status

MAPS (CAL ONLY)

Check the directories in VM (/var/www/maps/):

fp3d-content-a1-jh90-256_07.10.01.52391

fp3d-content-a2-jh90-256_07.10.00.51952

fp3d-content-a3-jh90-256_07.10.01.53802

fp3d-content-a4-jh90-256_07.10.01.53805

fp3d-content-c-cal-immfly_07.13.00.55806

fp3d-content-e-cal-immfly_07.13.00.55806

ADSB altitude simulation - ONLY for TEST

Run the following command in cd /p100_tools/adsb:

\$ bash adsb_simulator.sh raw_data_cal_01.txt

👤 Don't forget to add the labels to this document!

- coc
- tom
- ibs
- tui
- pgt
- voz
- voe
- wzz
- alert
- vara
- ify
- label1
- eqx
- vm-down
- libvirt